

Ateliers L3 CMI

Quentin Hoarau

2026-09-16

Table des matières

Introduction	5
Présentation	5
Notation	5
 I Général	 6
Bases de données	7
 II Logiciel Statistique et Econométrie	 8
Introduction	9
 TP1 : Premiers pas avec R et Quarto	 10
Projet RStudio et Quarto	10
Organiser un projet	10
Quarto	11
Vecteurs	13
Types, listes et tableaux	13
Packages, aide et chaînes de caractères	14
Rendu	14
 TP2 : Manipuler des tableaux de données	 15
Les verbes de base de <code>dplyr</code>	15
Importer et explorer des données	16
Agréger des données	17
Joindre et remodeler des tableaux	18
Rendu	18
 TP3 : Visualiser des données avec ggplot2	 19
La grammaire des graphiques	19
Séries temporelles	20
Bonnes pratiques de visualisation	21
Application : les émissions de CO2	22
Rendu	23

TP4 : Statistiques descriptives pour un projet	24
De la question aux données	24
Statistiques univariées	25
Statistiques bivariées	26
Tableau de statistiques et application au projet	26
Rendu	27
TP5 : Modèles de régression linéaire	28
Régression simple	28
Régression multiple	29
Forme fonctionnelle et variables qualitatives	30
Application : la courbe de Kuznets environnementale	30
Rendu	31
 III SIG	 32
Introduction	33
TP1 : Introduction à QGIS	34
1. Prise en main	34
1.1 Affichage / désaffichage des panneaux	34
1.2 Utilisation de données vecteur	34
1.3 Création d'un projet	35
1.4 Utilisation de l'outil Identifier les entités	35
1.5 Jointure 1	35
1.6 Utilisation d'OpenStreetMap	35
1.7 Ordre des couches et opacité	35
1.8 Groupe de couches	36
1.9 Outil de mesure	36
1.10 Sélection et export	36
1.11 Sélection et conditions multiples	36
2. Symbologie 1 : Pays du monde	36
2.1 Mise en page	37
2.2 Rendu gradué : Villes du monde	37
2.2 Symboles proportionnels	37
3. Symbologie 2 : France	37
3.1 Styles	38
3.2 Étiquettes	38
3.3 Mise en page finale	38
TP2 : Traitement sur les données vectorielles	39
1. Données de départ	39

2. Création d'un GeoPackage	39
3. Zone tampon sur l'autoroute A61	40
3.1 Vérification du SCR	40
3.2 Conversion en Lambert 93	40
3.3 Sélection et tampon	40
3.4 Analyse	40
4. Matrice de distance	41
5. Grille hexagonale	41
5.1 Création	41
5.2 Nettoyage	41
6. Comptages dans la grille	41
7. Analyse de superposition (Corine Land Cover)	42
7.1 Sélection des forêts	42
7.2 Superposition	42
8. Intersection et Group Stats	42
8.1 Zone tampon des parcs	42
8.2 Intersection avec CLC	42
8.3 Tableau croisé dynamique	43
TP3 : SIG avec R	44
1. Premières cartes	44
2. Opérations sur les attributs	45
3. Opération sur les données spatiales	46
3.1 Opérations sur les vecteurs	46
3.2 Opérations sur les rasters	47
4. Opérations sur les géométries	48
4.1 Opérations sur les vecteurs	48
4.2 Opérations sur les rasters	49
5. Application : rapprochement de base par distances	49

Introduction

Présentation

Ce livre contient les supports des trois ateliers spécifiques de la L3 CMI de l'université Paris Nanterre :

- Logiciel statistique et économétrie (S1) – 3 ECTS
- Système d'information géographique (S1) – 3 ECTS
- Econométrie 2 (S2) – 3 ECTS

La quasi-totalité des ateliers se font avec le langage R.

Notation

Toute la notation est en contrôle continu. Les notes viennent de :

- « Participation » qui inclut l'implication en classe, la qualité des rendus des TP et des pénalités pour retards et absences non justifiées
- Présentations en classe
- Projets de fin de semestre :
 - Peuvent être communs à plusieurs matières
 - Sujets donnés ou sujets libres
 - Rapport écrit propre (pas de compilation de code, ou de génération IA)
 - Présentation orale avec slides (sans les lire)

Tous les projets sont recensés dans ce [fichier Excel partagé](#).

Me contacter à qhoarau@parisnanterre.fr (attention : j'ai un homonyme à Nanterre).

Première partie

Général

Bases de données

Général :

- [datagouv](#) : répertoire public français
- [datagov US](#) : répertoire public américain
- [datagov UK](#) : répertoire public anglais
- [kaggle](#) : répertoire public pour data science
- [TidyTuesday](#) : projets collaboratifs de la [Data Science Learning Community](#).
- [Openintro](#) : données éducatives

Statistiques publiques :

- [DBnomics](#) ou voir le package R `dbnomics`
- [OCDE](#) ou voir le package R `ocde`
- [Eurostat](#) ou voir le package R `eurostat`

Données socioéconomiques :

- [Données du livre de Cagé & Piketty](#) : données à la maille commune sur les revenus, education, résultats électoraux...

Données SIG :

- [IGN](#)
- [Natural Earth](#)

Données Energie/environnement :

- [SDES](#) (Ministère Transition Ecologique).
- [ODRE](#) (réseaux d'énergie).
- [EEA](#) (Agence européenne de l'environnement).
- [Données incendies](#).
- [Données prix carburants](#).

Deuxième partie

Logiciel Statistique et Econométrie

Introduction

Le cours en résumé :

- 20 h, dont 4 h d'examen
- Objectifs : prise en main de R et Quarto, gestion et visualisation de données, statistiques descriptives, modèles de régression
- Langage : R
- Modalités d'examens :
 - Note de participation
 - Un projet avec choix du sujet libre
 - Présentation individuelle d'un chapitre d'un livre sur les bonnes pratiques de la dataviz : [Fundamentals of Data Visualization](#) de Claus Wilke. Inscription [ici](#).

Le plan :

1. Premiers pas avec R et Quarto
2. Manipuler des tableaux de données
3. Visualiser des données avec `ggplot2`
4. Statistiques descriptives pour un projet
5. Modèles de régression linéaire

En général pour R, regardez ce [bon tutoriel](#) sur le `tidyverse`.

TP1 : Premiers pas avec R et Quarto

```
knitr::opts_chunk$set(echo=FALSE, warning=FALSE, message=FALSE, eval=FALSE)

library(tidyverse)
library(here)
```

Ce TP présente l'environnement de travail utilisé pendant tout le semestre : l'organisation d'un projet, RStudio pour écrire et exécuter du code R, et Quarto pour rédiger des documents qui mêlent texte, code et résultats. Tous les TP suivants seront rendus sous la forme d'un document Quarto compilé en HTML.

Pour ce TP, cherchez les réponses par vous-même sans assistant IA : utilisez l'aide de R (`?nom_de_fonction`), un moteur de recherche, ou ce [tutoriel](#) sur R et le `tidyverse`.

Projet RStudio et Quarto

Organiser un projet

Un projet d'analyse de données accumule vite de nombreux fichiers : données, scripts, graphiques, rapport. Quelques règles permettent de s'y retrouver et de rendre le travail reproductible, c'est-à-dire exécutable sur un autre ordinateur sans modification :

- **un dossier par projet** (ici, un par TP), qui contient un fichier `.Rproj`. On ouvre toujours le projet en double-cliquant sur ce fichier : le répertoire de travail est alors la racine du projet ;
- **un sous-dossier par type de fichier** : `code/` pour les scripts `.R` et les documents `.qmd`, `data/` pour les données, `figures/` pour les graphiques exportés ;
- **les données brutes ne sont jamais modifiées à la main** : tous les traitements sont faits par le code, et les données transformées sont enregistrées sous un autre nom ;
- **des chemins relatifs** : jamais de chemin absolu (`C:/Users/moi/Bureau/...`) ni de `setwd()`, qui ne fonctionnent que sur votre ordinateur ;
- **des noms de fichiers sans espaces ni accents**, éventuellement numérotés dans l'ordre d'exécution (`01_import.R`, `02_graphiques.R`).

```
LSE/
+-- TP1/
|   +-- TP1.Rproj
|   +-- code/
|       |   +-- TP1.qmd
|   +-- data/
|   +-- figures/
+-- TP2/
    +-- TP2.Rproj
    +-- ...
```

Lors de la compilation, un document Quarto s'exécute depuis son propre dossier (ici `code/`) et non depuis la racine du projet. Pour écrire des chemins valables partout, on utilise la fonction `here()` du package `here`, qui construit le chemin à partir de la racine du projet : `here("data", "co2.csv")`.

Quarto

Un document Quarto (extension `.qmd`) comporte trois éléments :

- un **en-tête YAML**, délimité par `---`, qui contient les métadonnées du document (titre, auteur, format de sortie) ;
- du **texte** rédigé en Markdown ;
- des **chunks** de code R, délimités par trois accents graves, dont le résultat s'affiche dans le document.

```
---
title: "TP1 : Premiers pas avec R"
author: "Prénom Nom"
format: html
---

## Une section

Du texte en **gras**, en *italique*, et un [lien](https://quarto.org).

```{r}
1 + 1
```
```

La syntaxe Markdown de base est la suivante :

| Syntaxe | Résultat |
|--|-------------------------------|
| <code>## Titre, ### Sous-titre</code> | Titres de niveau 2 et 3 |
| <code>**gras**, *italique*</code> | gras , <i>italique</i> |
| <code>- élément</code> | Liste à puces |
| <code>1. élément</code> | Liste numérotée |
| <code>[texte] (https://...)</code> | Lien hypertexte |
| <code>`code`</code> | code |
| <code>\$y = \beta_0 + \beta_1 x\$</code> | $y = \beta_0 + \beta_1 x$ |

Les options d'un chunk s'écrivent en tête du chunk, précédées de `#|`. Par exemple, `#| echo: false` masque le code dans le document compilé.

- (i) Créer un dossier LSE sur votre drive. Dans RStudio, créer un projet TP1 dans ce dossier (File > New Project > New Directory > New Project). Quels fichiers sont créés ? Fermer RStudio, rouvrir le projet en double-cliquant sur TP1.Rproj, puis afficher le répertoire de travail dans la console avec `getwd()`.
- (ii) Créer les sous-dossiers `code`, `data` et `figures`, depuis l'onglet Files de RStudio ou avec `dir.create()`.
- (iii) Créer un document Quarto (File > New File > Quarto Document), en décochant l'éditeur visuel, et l'enregistrer sous `code/TP1.qmd`. Modifier l'en-tête YAML pour indiquer un titre, votre nom comme auteur et `format: html`. Compiler le document avec le bouton **Render** : où se trouve le fichier HTML produit ?
- (iv) Supprimer le contenu d'exemple. Rédiger une courte présentation de vous-même comprenant : un titre de section, un mot en gras et un mot en italique, une liste à puces de trois éléments, un lien vers le site quarto.org et l'équation $y = \beta_0 + \beta_1 x$. Compiler et vérifier le rendu. Ajouter ensuite une section pour chacune des parties suivantes du TP : dans la suite, répondre à chaque question dans un chunk et rédiger les réponses d'interprétation dans le texte.
- (v) Insérer un chunk (raccourci `Ctrl + Alt + I`, ou `Cmd + Option + I` sur Mac) qui calcule $2 + 3$. L'exécuter dans RStudio avec la flèche verte, puis compiler. Ajouter l'option `#| echo: false` et compiler de nouveau : que change-t-elle ? Même question avec `#| eval: false`.
- (vi) Ajouter un chunk contenant `getwd()` et compiler : quel répertoire s'affiche ? Comparer avec le résultat de la question (i). Installer le package `here` depuis la console, le charger dans le document, puis afficher `here("data", "co2.csv")`. Pourquoi ce chemin est-il préférable à `"C:/Users/moi/LSE/TP1/data/co2.csv"` ?

Vecteurs

En R, l'objet de base est le vecteur : une suite de valeurs de même type. On crée un objet avec l'opérateur d'assignation `<-`.

- (i) Créer le vecteur $x = (1, 2, 3, 4, 5)$ et le stocker dans un objet `x`, de trois façons : avec `c()`, avec `:` et avec `seq()`. Afficher sa classe avec `class()` et sa longueur avec `length()`.
- (ii) Extraire le premier élément de `x`, puis le dernier en utilisant `length()`. Stocker les trois premiers éléments dans un vecteur `a` et les éléments en positions 1, 3 et 5 dans un vecteur `b`.
- (iii) Calculer `(x + 10) * 2`, puis `a + b` et `a * b`. Calculer ensuite `x + a` : que se passe-t-il ?
- (iv) À l'aide de l'opérateur modulo `%`, afficher les éléments de `x` qui sont pairs. Afficher ensuite les éléments multiples de 2 **ou** de 3 (opérateur `|`), puis leurs positions avec `which()`.
- (v) Remplacer le premier élément de `x` par `NA`, puis calculer la somme et la moyenne de `x`. Consulter l'aide de `sum()` pour obtenir un résultat qui ignore la valeur manquante.
- (vi) Le tableau ci-dessous donne les émissions de CO2 par habitant en 2023, en tonnes (source : [Our World in Data](#)).

| Pays | France | Allemagne | États-Unis | Chine | Inde |
|------------|--------|-----------|------------|-------|------|
| CO2 / hab. | 4.1 | 7.0 | 14.3 | 8.6 | 2.1 |

Créer le vecteur `co2_hab` et nommer ses éléments avec `names()`. Calculer la moyenne, puis afficher le pays le plus émetteur avec `which.max()` et les pays dont les émissions dépassent la moyenne. Trier le vecteur par ordre décroissant avec `sort()`.

Types, listes et tableaux

Un vecteur ne contient qu'un seul type de valeurs (`numeric`, `character`, `logical`). Une liste peut contenir des objets de types différents. Un tableau de données (`data.frame` ou `tibble`) est une liste de vecteurs de même longueur : chaque colonne est une variable, chaque ligne une observation.

- (i) Évaluer `TRUE + FALSE + TRUE * 4`, puis `c(1, 4, TRUE)` et `c(1, 4, TRUE, "bonjour")`. Afficher la classe des deux derniers objets. Commenter.
- (ii) Évaluer `as.numeric("3.5")`, `as.numeric("trois")` et `as.character(12)`. Commenter.

- (iii) Créer une liste `moi` contenant trois éléments nommés `nom`, `prenom` et `annee_naissance`. Extraire votre prénom de deux façons : par sa position avec `[[ ]]`, puis par son nom avec `$`. Comparer la classe de `moi[2]` et celle de `moi[[2]]`.
- (iv) Ajouter à la liste `moi` un élément `formation`, puis afficher sa structure avec `str()`.
- (v) Créer avec `tibble()` un tableau `pays` contenant trois colonnes : le nom du pays, les émissions par habitant de la partie précédente et la population en millions d'habitants (France : 66.4 ; Allemagne : 84.5 ; États-Unis : 343.5 ; Chine : 1422.6 ; Inde : 1438.1). Afficher ses dimensions avec `dim()` et le nom de ses colonnes avec `names()`.
- (vi) Ajouter au tableau une colonne `co2_total` égale aux émissions totales du pays en millions de tonnes. Quel pays émet le plus au total ? Comparer avec le classement par habitant et commenter. Enregistrer enfin le tableau sous `data/pays.csv` avec `write_csv()` et `here()`.

Packages, aide et chaînes de caractères

Les packages étendent les fonctionnalités de R. On installe un package une seule fois avec `install.packages()`, puis on le charge à chaque session avec `library()`. Le **tidyverse** regroupe plusieurs packages, dont `stringr` pour manipuler les chaînes de caractères.

- (i) Installer le package `nycflights13`, qui sera utilisé au prochain TP, puis le charger. Pourquoi ne faut-il pas laisser `install.packages()` dans un document Quarto ?
- (ii) Ouvrir l'aide de la fonction `mean()`. Quels sont ses arguments ? Que fait l'argument `trim` ?
- (iii) Créer les objets `a` et `b` contenant respectivement les chaînes « 23 à 0 » et « C'est la piquette, Jack! », puis le vecteur `phrases` contenant `a` et `b`. Afficher le nombre de caractères de chaque élément avec `str_length()`.
- (iv) Concaténer `a` et `b` avec `str_c()`, en les séparant par une virgule. Mettre les éléments de `phrases` en majuscules avec `str_to_upper()`.
- (v) Tester avec `str_detect()` si le motif « piqu » est présent dans les éléments de `phrases`. Remplacer toutes les lettres « a » de `b` par « Z » avec `str_replace_all()`.
- (vi) Un fichier de données contient les noms de pays suivants, mal saisis : `c(" france", "ALLEMAGNE ", "etats-unis", "chine ")`. Les nettoyer pour obtenir « France », « Allemagne », « Etats-Unis » et « Chine », à l'aide de `str_trim()` et `str_to_title()`.

Rendu

Compiler le document complet. Il doit se compiler sans erreur après un redémarrage de R (`Session > Restart R`) : cela garantit qu'il ne dépend pas d'objets créés à la main dans la console. Déposer le fichier `TP1.html` sur CEL.

TP2 : Manipuler des tableaux de données

```
knitr::opts_chunk$set(echo=FALSE, warning=FALSE, message=FALSE, eval=FALSE)

library(tidyverse)
library(here)
library(nycflights13)
```

Ce TP présente les principales opérations sur les tableaux de données avec les packages **dplyr** et **tidyr** du **tidyverse** : sélectionner, filtrer, trier, créer des variables, agréger, joindre et remodeler. On travaille d'abord sur les vols au départ de New York, puis sur les émissions de CO2 des pays du monde.

Comme au TP1, créer un projet RStudio TP2 avec les sous-dossiers **code**, **data** et **figures**, et rédiger les réponses dans le document **code/TP2.qmd**. Le [chapitre sur dplyr](#) du tutoriel de Julien Barnier est une bonne référence.

Les verbes de base de dplyr

Le package **nycflights13** contient plusieurs tables sur les vols au départ des trois aéroports de New York en 2013.

| Table / variable | Description |
|-----------------------------|--|
| flights | Un vol par ligne |
| month, day | Mois et jour du vol |
| dep_delay, arr_delay | Retard au départ et à l'arrivée (en minutes) |
| carrier | Code de la compagnie aérienne |
| origin, dest | Codes des aéroports de départ et de destination |
| airports | Un aéroport par ligne : code faa , nom name , altitude alt (en pieds), lat , lon , fuseau horaire tz et tzone |
| airlines | Une compagnie par ligne : code carrier et nom name |

Chaque verbe de **dplyr** prend un tableau en premier argument et renvoie un tableau. Le *pipe* **%>%** permet d'enchaîner les opérations : **tableau %>% filter(...)** **%>% select(...)** se lit « prendre le tableau, puis filtrer, puis sélectionner ».

- (i) Afficher la structure de la table `flights` avec `glimpse()`. Combien compte-t-elle de lignes et de colonnes ? Que représente une ligne ?
- (ii) Avec `filter()`, sélectionner les vols du mois de juillet, puis les vols dont le retard à l'arrivée est compris entre 5 et 15 minutes, puis les vols des compagnies Delta, United et American (codes DL, UA et AA, utiliser `%in%`). Combien de vols obtient-on dans chaque cas ?
- (iii) Trier la table `flights` par retard au départ décroissant avec `arrange()` et `desc()`. Quel est le plus long retard ? Afficher l'aéroport situé à l'altitude la plus basse avec `slice_min()`.
- (iv) Dans la table `airports`, sélectionner avec `select()` les colonnes `name`, `lat` et `lon`, puis toutes les colonnes sauf `tz` et `tzone`. Dans `flights`, sélectionner les colonnes dont le nom se termine par `delay` (`ends_with()`). Renommer enfin la colonne `alt` de `airports` en `altitude` avec `rename()`.
- (v) Avec `mutate()`, créer dans `airports` une variable `alt_m` donnant l'altitude en mètres (1 mètre = 3.2808 pieds), puis ne conserver que `name`, `alt` et `alt_m`. Écrire ces opérations avec le *pipe*.
- (vi) En une seule chaîne d'opérations : sélectionner les vols de septembre et d'octobre, conserver les colonnes `dest` et `dep_delay`, créer une variable `retard_h` donnant le retard au départ en heures, et afficher les 5 vols les plus retardés avec `slice_max()`.

Importer et explorer des données

On utilise la base de données sur les émissions de CO2 de [Our World in Data](https://ourworldindata.org/), qui couvre tous les pays depuis 1750. Les variables utilisées sont les suivantes :

| Variable | Description |
|-----------------------------|--|
| <code>country</code> | Pays ou agrégat régional |
| <code>year</code> | Année |
| <code>iso_code</code> | Code ISO du pays à 3 lettres (vide pour les agrégats régionaux) |
| <code>population</code> | Population |
| <code>gdp</code> | PIB, en dollars internationaux de 2011 (parité de pouvoir d'achat) |
| <code>co2</code> | Émissions annuelles de CO2, hors changement d'usage des sols (en millions de tonnes) |
| <code>co2_per_capita</code> | Émissions de CO2 par habitant (en tonnes) |

- (i) Dans la console, télécharger le fichier dans le dossier `data` avec `download.file()` :

```
download.file("https://raw.githubusercontent.com/owid/co2-data/master/owid-co2-data.csv",
             here("data", "owid-co2-data.csv"))
```

Dans le document, importer ensuite le fichier avec `read_csv()` et `here()` dans un objet `co2`. Pourquoi télécharger le fichier une fois plutôt que lire directement l'URL à chaque compilation ?

- (ii) Afficher les dimensions de `co2` et sa structure. Que représente une ligne du tableau ?
- (iii) Quelle période couvrent les données ? Combien de pays ou régions distincts contiennent-elles (utiliser `n_distinct()`) ?
- (iv) Afficher la liste des entités dont le code ISO est manquant (`is.na()`, puis `distinct()`). De quoi s'agit-il ? Pourquoi faut-il les retirer avant de calculer des totaux ?
- (v) Créer un tableau `co2_pays` qui ne contient que les pays (code ISO renseigné), à partir de 1990, et les seules variables du tableau ci-dessus. Afficher un résumé du tableau avec `summary()`, puis les données de la France de 2020 à 2024. Que remarquez-vous pour la variable `gdp` ?
- (vi) Enregistrer `co2_pays` dans le dossier `data` avec `write_csv()`, sous un nom différent du fichier d'origine.

Agréger des données

Pour résumer un tableau par groupe, on combine `group_by()`, qui définit les groupes, et `summarise()`, qui calcule une statistique par groupe. La fonction `count()` compte le nombre de lignes par groupe.

- (i) Compter le nombre de vols par mois avec `count()`, puis trier le résultat par nombre de vols croissant. Quel est le mois le plus chargé ?
- (ii) Calculer, pour chaque aéroport de départ (`origin`), le retard moyen au départ et le nombre de vols (`n()`). Pourquoi faut-il utiliser l'argument `na.rm = TRUE` ?
- (iii) Pour chaque mois, afficher la destination qui compte le plus de vols. Utiliser `count()`, `group_by()` et `slice_max()`.
- (iv) À partir de `co2_pays`, calculer les émissions mondiales de CO2 pour chaque année. Comparer la valeur de 2023 avec celle de la ligne `World` du tableau `co2`. D'où peut venir l'écart ?
- (v) Pour l'année 2023, calculer la part de chaque pays dans les émissions mondiales, puis afficher les 10 plus gros émetteurs avec leurs émissions totales, leurs émissions par habitant et leur part. Commenter.
- (vi) Pour la France, l'Allemagne, les États-Unis, la Chine et l'Inde, créer une variable `decennie` (utiliser la division entière `%/%`) et calculer les émissions moyennes par habitant par décennie. Commenter les évolutions.

Joindre et remodeler des tableaux

Une jointure associe les lignes de deux tableaux à partir d'une variable commune, appelée clé. `left_join(x, y)` conserve toutes les lignes de `x` et leur ajoute les colonnes de `y`; `anti_join(x, y)` renvoie les lignes de `x` sans correspondance dans `y`.

Un tableau est au format **long** lorsque chaque ligne est une observation (un pays une année), et au format **large** lorsque les modalités d'une variable sont en colonnes (une colonne par année). On passe de l'un à l'autre avec `pivot_wider()` et `pivot_longer()`.

- (i) Joindre la table `airlines` à la table `flights` avec `left_join()`. Calculer ensuite le retard moyen au départ pour chaque compagnie, en utilisant son nom, et trier par retard décroissant.
- (ii) Joindre la table `airports` à la table `flights` en utilisant comme clé le code de l'aéroport de destination (`by = c("dest" = "faa")`). Quelles sont les trois destinations les plus fréquentes? Avec `anti_join()`, identifier les destinations absentes de la table `airports`.
- (iii) Importer la [classification des pays par continent](https://ourworldindata.org/grapher/continents-according-to-our-world-in-data) de Our World in Data, en la téléchargeant d'abord dans `data` comme à la partie précédente :

```
download.file("https://ourworldindata.org/grapher/continents-according-to-our-world-in-data.  
here("data", "continents.csv"))
```

Joindre la variable `owid_region`, renommée `continent`, au tableau `co2_pays` : la clé est `iso_code` d'un côté et `code` de l'autre. Quels pays n'ont pas de continent?

- (iv) Calculer les émissions totales de chaque continent en 1990 et en 2023, puis passer au format large avec `pivot_wider()` (une colonne par année). Ajouter une variable donnant l'évolution en pourcentage entre les deux années. Commenter.
- (v) Revenir au format long à partir du tableau précédent avec `pivot_longer()`. Pourquoi le format long est-il le plus adapté pour faire des calculs par groupe ou des graphiques?
- (vi) Enregistrer le tableau `co2_pays` complété des continents dans `data/co2_continents.csv`. Il sera réutilisé au TP3.

Rendu

Compiler le document complet. Il doit se compiler sans erreur après un redémarrage de R (`Session > Restart R`). Déposer le fichier `TP2.html` sur CEL.

TP3 : Visualiser des données avec ggplot2

```
knitr::opts_chunk$set(echo=FALSE, warning=FALSE, message=FALSE, eval=FALSE)

library(tidyverse)
library(here)
library(questionr)
library(scales)
```

Ce TP présente la construction de graphiques avec le package **ggplot2** : la logique de la grammaire des graphiques, les séries temporelles, les bonnes pratiques de visualisation, puis une application aux émissions de CO2.

Comme aux TP précédents, créer un projet RStudio TP3 avec les sous-dossiers **code**, **data** et **figures**, et rédiger les réponses dans **code/TP3.qmd**. Pour aller plus loin : le [chapitre sur ggplot2](#) du tutoriel de Julien Barnier et le livre [Fundamentals of Data Visualization](#) de Claus Wilke.

La grammaire des graphiques

Un graphique **ggplot2** se construit par couches, ajoutées avec **+** :

- **ggplot(data)** indique le tableau de données ;
- une fonction **geom_*()** indique la forme géométrique : points (**geom_point**), barres (**geom_bar**), boîtes à moustaches (**geom_boxplot**), lignes (**geom_line**), etc. ;
- la fonction **aes()** associe des variables du tableau à des propriétés visuelles : position (**x**, **y**), couleur (**color**, **fill**), taille (**size**), transparence (**alpha**).

On utilise le jeu de données **rp2018** du package **questionr**, issu du recensement de la population 2018 de l'Insee. Il contient une ligne par commune de plus de 2 000 habitants. Les pourcentages sont exprimés en points de pourcentage.

| Variable | Description |
|--------------------|-------------------|
| commune | Nom de la commune |
| departement | Département |

| Variable | Description |
|-------------------------|---|
| <code>pop_tot</code> | Population totale |
| <code>pop_cl</code> | Taille de la commune, en classes |
| <code>dipl_aucun</code> | Part des personnes sans diplôme (%) |
| <code>dipl_sup</code> | Part des diplômés du supérieur (%) |
| <code>ouvr</code> | Part des ouvriers (%) |
| <code>cadres</code> | Part des cadres (%) |
| <code>chom</code> | Part des chômeurs dans la population active (%) |
| <code>proprio</code> | Part des ménages propriétaires de leur logement (%) |

- (i) Charger le jeu de données avec `data(rp2018)` et créer un tableau `rp69` qui ne contient que les communes du Rhône et de la Loire. Représenter le pourcentage d'ouvriers (`ouvr`) en fonction du pourcentage de personnes sans diplôme (`dipl_aucun`) par un nuage de points. Commenter.
- (ii) Refaire le nuage de points avec des points rouges et une transparence de 0.2. Faire ensuite varier la couleur des points selon le département et leur taille selon la population. Quelle est la différence entre un argument placé dans `aes()` et un argument placé en dehors ?
- (iii) Représenter la distribution du pourcentage de propriétaires selon la taille de la commune par des boîtes à moustaches, avec une couleur de remplissage (`fill`) par département. Commenter.
- (iv) Représenter le nombre de communes par classe de taille avec `geom_bar()`, avec une couleur de remplissage par département. Modifier ensuite l'argument `position` de `geom_bar()` avec les valeurs "dodge" puis "fill". Que représente l'axe des ordonnées avec `position = "fill"` ?
- (v) Représenter le pourcentage de diplômés du supérieur en fonction du pourcentage de cadres, avec un graphique par département (`facet_wrap()`). Ajouter une droite de régression avec `geom_smooth(method = "lm")`. Commenter.
- (vi) Représenter le pourcentage de personnes sans diplôme en fonction du taux de chômage. Afficher en rouge le nom des communes dont le taux de chômage dépasse 15 % avec `geom_text()`, en lui passant un tableau filtré dans l'argument `data`.

Séries temporelles

On utilise le jeu de données `economics`, fourni avec `ggplot2`, qui contient des séries mensuelles de l'économie américaine de 1967 à 2015.

| Variable | Description |
|-------------------|-------------|
| <code>date</code> | Mois |

| Variable | Description |
|----------------------|--|
| <code>pce</code> | Dépenses de consommation des ménages (en milliards de dollars) |
| <code>psavert</code> | Taux d'épargne des ménages (%) |
| <code>uempmed</code> | Durée médiane du chômage (en semaines) |

- (i) Représenter l'évolution des dépenses de consommation (`pce`) avec `geom_line()`, en bleu ("`dodgerblue`") et avec une épaisseur de ligne (`linewidth`) de 0.5. Stocker le graphique dans un objet `p_1`.
- (ii) Ajouter à `p_1` un titre et retirer les titres des axes avec `labs()`. Afficher une graduation tous les 5 ans avec seulement l'année, grâce aux arguments `date_breaks` et `date_labels` de `scale_x_date()`.
- (iii) Sélectionner les variables `date`, `psavert` et `uempmed`, puis passer au format long avec `pivot_longer()` : le tableau `df` obtenu doit contenir les colonnes `date`, `variable` et `valeur`. Représenter les deux séries sur un même graphique, avec une couleur par série.
- (iv) Différencier les séries à la fois par la couleur et le type de ligne (`linetype`). Avec `scale_color_manual()` et `scale_linetype_manual()`, afficher le taux d'épargne en bleu et en tirets, la durée du chômage en rouge et en trait plein, avec des étiquettes en français (argument `labels`). Placer la légende en bas, sans titre, et appliquer le thème `theme_minimal()`.
- (v) Les deux séries n'ont pas la même unité. Pourquoi est-ce un problème de les représenter sur le même axe ? Représenter chaque série dans un panneau séparé avec `facet_wrap()` et l'argument `scales = "free_y"`. Commenter les évolutions.
- (vi) Enregistrer le graphique précédent au format PDF dans le dossier `figures`, avec `ggsave()` et `here()`, en largeur 20 cm et hauteur 12 cm.

Bonnes pratiques de visualisation

Un bon graphique transmet un message clair sans effort de lecture. Quelques règles, tirées du livre de Claus Wilke :

- un titre informatif, des axes avec leurs unités et la source des données ;
- des catégories ordonnées selon la valeur représentée plutôt que par ordre alphabétique ;
- pas de couleur ni de légende qui n'apportent pas d'information ;
- une échelle logarithmique pour les variables très asymétriques (population, revenu) ;
- des palettes adaptées : séquentielles pour des catégories ordonnées, qualitatives sinon, et lisibles par les daltoniens ;
- pas d'effets 3D, de camemberts à nombreuses parts ni de double axe des ordonnées.

Le code suivant produit un graphique du taux de chômage par région :

```
rp2018 %>%
  group_by(region) %>%
  summarise(chom = mean(chom)) %>%
  ggplot() +
  geom_col(aes(x = region, y = chom, fill = region))
```

- (i) Reproduire ce graphique et lister ses défauts.
- (ii) Corriger le graphique : barres horizontales, régions triées par taux de chômage (`fct_reorder()`), suppression de la couleur, titre, titres d'axes avec unités et source (argument `caption` de `labs()`), thème `theme_minimal()`.
- (iii) Le calcul lui-même pose problème : la moyenne des taux communaux donne le même poids à une commune de 2 000 habitants et à Marseille. Recalculer le taux de chômage de chaque région comme le rapport entre le nombre total de chômeurs (`pop_chom`) et le nombre total d'actifs (`pop_act_15p`). Comparer les deux indicateurs et expliquer l'écart.
- (iv) Représenter la distribution de la population des communes (`pop_tot`) par un histogramme. Pourquoi est-il illisible? Utiliser une échelle logarithmique avec `scale_x_log10()`, et des étiquettes lisibles avec `labels = label_number(big.mark = " ")`.
- (v) Pour l'ensemble des communes, représenter le pourcentage de diplômés du supérieur en fonction du pourcentage de cadres, avec une couleur par classe de taille de commune. Utiliser la palette `scale_color_viridis_d()`. Pourquoi une palette séquentielle est-elle adaptée ici?

Application : les émissions de CO2

On reprend les données d'Our World in Data utilisées au TP2 (voir le tableau des variables du TP2).

- (i) Copier dans le dossier `data` les fichiers `owid-co2-data.csv` et `continents.csv` téléchargés au TP2. Les importer et les joindre comme au TP2, en ne conservant que les pays rattachés à un continent, mais sans restreindre la période. Stocker le résultat dans un tableau `co2_pays`.
- (ii) Représenter l'évolution des émissions de CO2 par habitant depuis 1950 pour la France, l'Allemagne, les États-Unis, la Chine et l'Inde, avec une couleur par pays. Ajouter un titre, l'unité et la source. Commenter.
- (iii) Représenter l'évolution des émissions totales de chaque continent depuis 1950 par un graphique en aires empilées (`geom_area()`). Commenter.
- (iv) Pour l'année 2022, calculer le PIB par habitant et représenter les émissions par habitant en fonction du PIB par habitant, avec une échelle logarithmique sur les deux axes. La taille des points dépend de la population et leur couleur du continent. Commenter.

- (v) Ajouter au graphique précédent le nom de quelques pays (France, États-Unis, Chine, Inde, Qatar) avec `geom_text()`, puis l'enregistrer au format PNG dans le dossier **figures**.
- (vi) Dans le document Quarto, ajouter au chunk du graphique précédent les options `#| label: fig-co2-pib` et `#| fig-cap:` suivie d'une légende. Écrire une phrase qui fait référence à la figure avec `@fig-co2-pib`. Compiler : que se passe-t-il ?

Rendu

Compiler le document complet. Il doit se compiler sans erreur après un redémarrage de R (`Session > Restart R`). Déposer le fichier `TP3.html` sur CEL.

TP4 : Statistiques descriptives pour un projet

```
knitr::opts_chunk$set(echo=FALSE, warning=FALSE, message=FALSE, eval=FALSE)

library(tidyverse)
library(here)
library(questionr)
library(stargazer)
```

Toute étude empirique commence par une description soignée des données : qui est observé, quelles variables sont disponibles, quelle est leur qualité, et quelles relations apparaissent entre elles. Ce TP suit cette démarche sur une enquête de l’Insee, puis l’applique aux données de votre projet.

Comme aux TP précédents, créer un projet RStudio TP4 avec les sous-dossiers `code`, `data` et `figures`, et rédiger les réponses dans `code/TP4.qmd`.

De la question aux données

On cherche à répondre à deux questions : **qui pratique une activité sportive ?** et **qui regarde le plus la télévision ?** On utilise le jeu de données `hdv2003` du package `questionr`, un extrait de l’enquête *Histoire de vie* réalisée par l’Insee en 2003 auprès de 2 000 personnes.

| Variable | Description |
|------------------------|---|
| <code>id</code> | Identifiant de l’enquêté |
| <code>age</code> | Âge (en années) |
| <code>sexe</code> | Sexe |
| <code>nivetud</code> | Niveau d’études le plus élevé atteint |
| <code>occup</code> | Occupation actuelle (emploi, chômage, études, retraite, etc.) |
| <code>qualif</code> | Qualification professionnelle |
| <code>sport</code> | Pratique d’une activité sportive (Oui / Non) |
| <code>heures.tv</code> | Nombre d’heures passées devant la télévision par jour |
| <code>poids</code> | Poids de sondage |

- (i) Charger le jeu de données avec `data(hdv2003)`. Quelle est l'unité d'observation ? Parmi les variables du tableau ci-dessus, lesquelles sont quantitatives et lesquelles sont qualitatives ?
- (ii) Compter le nombre de valeurs manquantes de chaque variable avec `summarise()`, `across()` et `is.na()`. Quelles variables ont beaucoup de valeurs manquantes ? Proposer une explication pour `qualif`.
- (iii) Afficher l'occupation et l'âge moyen des personnes dont le niveau d'études est manquant. À quoi correspondent ces valeurs manquantes ?
- (iv) Créer un tableau `hdv` avec une variable `etudes` qui regroupe `nivetud` en quatre modalités avec `fct_collapse()` : « Primaire ou moins », « Secondaire », « Technique ou professionnel » et « Supérieur ». Transformer ensuite les valeurs manquantes en une modalité « En cours d'études » avec `fct_na_value_to_level()`. Vérifier le résultat avec `count()`.
- (v) Ajouter au tableau `hdv` une variable `age_cl` en classes d'âge (18-29, 30-44, 45-59, 60-74, 75 et plus) avec `cut()`, et une variable `statut` qui distingue les actifs occupés, les chômeurs et les inactifs avec `case_when()`.
- (vi) Vérifier qu'il n'y a pas de doublons (un `id` par ligne). Repérer les valeurs extrêmes de `heures.tv` : combien de personnes déclarent plus de 8 heures par jour ? Faut-il les supprimer ?

Statistiques univariées

On décrit chaque variable séparément : tendance centrale (moyenne, médiane) et dispersion (écart-type, quantiles) pour les variables quantitatives, tableau de fréquences pour les variables qualitatives.

- (i) Calculer la moyenne, la médiane, l'écart-type, le minimum et le maximum de `age` et de `heures.tv`. Afficher les déciles de `heures.tv` avec `quantile()`.
- (ii) Obtenir le même résultat en une seule instruction avec `summarise()` et `across()`, en passant une liste de fonctions : `list(moyenne = ~ mean(.x, na.rm = TRUE), ...)`.
- (iii) Représenter la distribution de `age` par un histogramme (`binwidth = 5`) et celle de `heures.tv` par un diagramme en barres. Comparer la moyenne et la médiane de `heures.tv` et commenter la forme de sa distribution.
- (iv) Calculer les effectifs et les pourcentages des modalités de `sexe`, `etudes`, `statut` et `sport`.
- (v) L'échantillon n'est pas parfaitement représentatif de la population française : chaque personne a un poids de sondage (`poids`). Comparer la part de sportifs dans l'échantillon et la part pondérée, calculée avec `weighted.mean()`. Laquelle faut-il retenir pour décrire la population française ?

Statistiques bivariées

On étudie maintenant les relations entre deux variables. L'outil dépend de leur nature : tableau croisé et test du χ^2 pour deux variables qualitatives, moyennes par groupe et test de Student pour une variable quantitative et une variable qualitative, corrélation pour deux variables quantitatives.

- (i) Construire le tableau croisé du niveau d'études et de la pratique sportive, avec les pourcentages en ligne : pour chaque niveau d'études, quelle part des personnes pratique un sport ? Utiliser `count()`, `group_by()` et `mutate()`. Commenter.
- (ii) Tester l'indépendance entre niveau d'études et pratique sportive avec `chisq.test()`, appliqué à `table(hdv$etudes, hdv$sport)`. Quelle est l'hypothèse nulle ? Conclure.
- (iii) Calculer le nombre moyen d'heures de télévision des hommes et des femmes. La différence est-elle significative ? Utiliser `t.test(heures.tv ~ sexe, data = hdv)`.
- (iv) Représenter la distribution de `heures.tv` selon le niveau d'études par des boîtes à moustaches, et calculer la moyenne par niveau. Commenter.
- (v) Calculer la corrélation entre l'âge et le nombre d'heures de télévision avec `cor()` (argument `use = "complete.obs"`), puis tester sa significativité avec `cor.test()`. Commenter.
- (vi) Calculer la part de sportifs par classe d'âge, puis l'âge moyen par niveau d'études. Peut-on conclure de la question (i) que faire des études augmente la pratique sportive ?

Tableau de statistiques et application au projet

Un rapport présente les statistiques descriptives dans des tableaux soignés plutôt que dans des sorties brutes de R. Dans un document Quarto, un tableau produit par `stargazer()` s'affiche avec l'argument `type = "html"` et l'option de chunk `## results: asis`.

- (i) Produire un tableau de statistiques descriptives de `age`, `heures.tv` et `freres.soeurs` avec `stargazer()`. La fonction attend un `data.frame` : utiliser `as.data.frame()`. Tester `type = "text"` dans la console, puis `type = "html"` dans le document compilé.
- (ii) Construire un tableau qui donne, pour chaque niveau d'études, l'effectif, l'âge moyen, le nombre moyen d'heures de télévision et la part de sportifs. L'afficher avec `knitr::kable()`, en arrondissant à une décimale (`digits`) et avec des noms de colonnes lisibles (`col.names`).
- (iii) Rédiger dans le document un paragraphe de description de l'échantillon, dont les chiffres sont calculés par du code en ligne. Par exemple, la phrase suivante affiche l'âge moyen calculé au moment de la compilation :

```
L'âge moyen est de `r round(mean(hdv$age), 1)` ans.
```

Quel est l'avantage par rapport à un chiffre recopié à la main ?

- (iv) Pour votre projet : formuler la question de recherche, puis importer vos données dans le dossier **data**. Préciser l'unité d'observation, la période et le champ couverts, la variable d'intérêt et trois à cinq variables explicatives. Faire le bilan des valeurs manquantes et effectuer les recodages nécessaires.
- (v) Toujours pour votre projet : produire les statistiques univariées et bivariées pertinentes pour votre question (tableaux et graphiques), un tableau de statistiques descriptives, et rédiger une description des données d'une dizaine de lignes.

Rendu

Compiler le document complet. Il doit se compiler sans erreur après un redémarrage de R (`Session > Restart R`). Déposer le fichier `TP4.html` sur CEL.

TP5 : Modèles de régression linéaire

```
knitr::opts_chunk$set(echo=FALSE, warning=FALSE, message=FALSE, eval=FALSE)

library(tidyverse)
library(here)
library(fixest)
library(wooldridge)
library(questionr)
```

Ce TP présente l'estimation et l'interprétation des modèles de régression linéaire par les moindres carrés ordinaires (MCO) : régression simple, régression multiple, forme fonctionnelle et variables qualitatives, puis une application à la relation entre émissions de CO2 et niveau de vie.

Comme aux TP précédents, créer un projet RStudio TP5 avec les sous-dossiers `code`, `data` et `figures`, et rédiger les réponses dans `code/TP5.qmd`. Les régressions sont estimées avec la fonction `feols()` du package `fixest`, et comparées dans des tableaux produits par `etable()`.

Régression simple

On étudie le rendement de l'éducation à partir du jeu de données `wage1` du package `wooldridge` : 526 salariés américains interrogés en 1976.

| Variable | Description |
|---------------------|--|
| <code>wage</code> | Salaire horaire moyen (en dollars) |
| <code>lwage</code> | Logarithme du salaire horaire |
| <code>educ</code> | Nombre d'années d'études |
| <code>exper</code> | Nombre d'années d'expérience professionnelle potentielle |
| <code>tenure</code> | Nombre d'années d'ancienneté chez l'employeur actuel |
| <code>female</code> | Vaut 1 si la personne est une femme, 0 sinon |

Le modèle de régression simple s'écrit :

$$wage_i = \beta_0 + \beta_1 educ_i + u_i$$

- (i) Charger le jeu de données avec `data(wage1)`. Calculer le salaire horaire moyen et le nombre moyen d'années d'études. Représenter le salaire en fonction du nombre d'années d'études par un nuage de points, en ajoutant la droite de régression avec `geom_smooth(method = "lm")`.
- (ii) Estimer le modèle avec `feols(wage ~ educ, data = wage1)` et afficher les résultats avec `summary()`. Interpréter $\hat{\beta}_1$ et $\hat{\beta}_0$. La constante a-t-elle un sens économique ?
- (iii) Le coefficient $\hat{\beta}_1$ est-il significativement différent de zéro au seuil de 5 % ? Afficher son intervalle de confiance à 95 % avec `confint()` et l'interpréter.
- (iv) Quelle part de la variance des salaires le modèle explique-t-il ? Utiliser `r2()`.
- (v) Calculer avec `predict()` le salaire prédit pour une personne ayant 12 années d'études (fin du lycée), puis 16 années (licence). Ajouter au tableau les valeurs prédites (`fitted()`) et les résidus (`resid()`), puis vérifier que la moyenne des résidus est nulle.

Régression multiple

La régression multiple permet d'isoler l'effet d'une variable en contrôlant les autres : chaque coefficient s'interprète « toutes choses égales par ailleurs ». Si une variable qui influence y et qui est corrélée à une variable explicative est omise, le coefficient de cette dernière est biaisé : c'est le **biais de variable omise**.

$$wage_i = \beta_0 + \beta_1 educ_i + \beta_2 exper_i + \beta_3 tenure_i + u_i$$

- (i) Estimer le modèle ci-dessus. Interpréter chacun des coefficients.
- (ii) Comparer les résultats des régressions simple et multiple dans un tableau avec `etable()`. Comment évolue le coefficient de `educ` ? Calculer la corrélation entre `educ` et `exper`, puis expliquer cette évolution par le biais de variable omise.
- (iii) Comparer le R^2 et le R^2 ajusté des deux modèles. Pourquoi le R^2 ajusté est-il préférable pour comparer des modèles qui n'ont pas le même nombre de variables ?
- (iv) Tester l'hypothèse nulle $H_0 : \beta_2 = \beta_3 = 0$ avec la fonction `wald()`, argument `keep = "exper|tenure"`. Conclure.
- (v) Peut-on interpréter $\hat{\beta}_1$ comme l'effet causal d'une année d'études supplémentaire sur le salaire ? Donner un exemple de variable omise qui pourrait encore biaiser ce coefficient, et le sens probable du biais.

Forme fonctionnelle et variables qualitatives

Le modèle linéaire peut intégrer des relations non linéaires grâce à des transformations des variables :

| Modèle | Équation | Interprétation de β_1 |
|---------------|---------------------------------------|---|
| Niveau-niveau | $y = \beta_0 + \beta_1 x$ | x augmente d'une unité : y varie de β_1 unités |
| Log-niveau | $\log(y) = \beta_0 + \beta_1 x$ | x augmente d'une unité : y varie d'environ $100\beta_1$ % |
| Log-log | $\log(y) = \beta_0 + \beta_1 \log(x)$ | x augmente de 1 % : y varie de β_1 % (élasticité) |

Une variable indicatrice (0/1) mesure l'écart moyen entre deux groupes, à autres variables égales. Pour une variable qualitative à plusieurs modalités, `feols()` crée automatiquement une indicatrice par modalité, sauf pour la modalité de référence (la première). Lorsque la variable expliquée vaut 0 ou 1, un coefficient s'interprète comme une variation de probabilité, en points de pourcentage.

- (i) Estimer le modèle précédent en remplaçant `wage` par `lwage`. Interpréter le coefficient de `educ`.
- (ii) Créer une variable `exper2` égale au carré de l'expérience et l'ajouter au modèle de la question (i). Interpréter les signes des coefficients de `exper` et `exper2`. Calculer le nombre d'années d'expérience à partir duquel le salaire prédit commence à baisser : $-\beta_{exper}/(2\beta_{exper2})$. Quelle part de l'échantillon se trouve au-delà ?
- (iii) Ajouter la variable `female` au modèle. Interpréter son coefficient, puis le comparer avec l'écart de salaire brut entre femmes et hommes, obtenu en régressant `lwage` sur `female` seulement. Commenter.
- (iv) Présenter les trois modèles en log dans un même tableau avec `etable()`, en donnant des noms lisibles aux variables grâce à l'argument `dict`.
- (v) On revient aux données `hdv2003` du TP4. Reprendre le code du TP4 pour créer la variable `etudes`, et créer une variable `sportif` qui vaut 1 si la personne pratique un sport et 0 sinon. Régresser `sportif` sur `etudes`. Interpréter la constante et le coefficient de la modalité « Supérieur ». Retrouve-t-on les résultats du TP4 ?
- (vi) Ajouter l'âge, puis le sexe au modèle. Comment évolue le coefficient de « Supérieur » ? Que peut-on conclure sur le lien entre études et pratique sportive ?

Application : la courbe de Kuznets environnementale

Selon l'hypothèse de la **courbe de Kuznets environnementale** (Grossman et Krueger, 1995), la pollution augmente avec le niveau de vie aux premiers stades du développement,

puis diminue au-delà d'un certain revenu, grâce aux changements de structure productive, aux technologies plus propres et à la demande de qualité environnementale. La relation entre émissions et revenu prendrait la forme d'un U inversé :

$$co2_hab_i = \beta_0 + \beta_1 pib_hab_i + \beta_2 pib_hab_i^2 + u_i$$

avec $\beta_1 > 0$ et $\beta_2 < 0$. Le revenu à partir duquel les émissions diminuent est alors $pib^* = -\beta_1/(2\beta_2)$. On teste cette hypothèse en coupe transversale sur les pays du monde en 2022, avec les données d'Our World in Data utilisées aux TP2 et TP3.

- (i) Copier dans le dossier `data` les fichiers `owid-co2-data.csv` et `continents.csv` des TP précédents. Construire un tableau `co2_2022` qui contient les pays (rattachés à un continent) en 2022, avec le PIB par habitant en milliers de dollars (`pib_hab`), son carré (`pib_hab2`) et les émissions par habitant, en retirant les observations manquantes.
- (ii) Estimer le modèle linéaire `co2_per_capita ~ pib_hab`. Interpréter le coefficient et le R^2 .
- (iii) Estimer le modèle quadratique ci-dessus. Les signes et la significativité des coefficients sont-ils compatibles avec une courbe de Kuznets ?
- (iv) Représenter le nuage de points avec la courbe estimée : `geom_smooth(method = "lm", formula = y ~ x + I(x^2))`. Quel pays se distingue ? Ré-estimer le modèle quadratique sans ce pays et calculer pib^* . Combien de pays ont un PIB par habitant supérieur à pib^* ? Que conclure sur la robustesse du résultat ?
- (v) Estimer le modèle en log-log : $\log(co2_hab) = \beta_0 + \beta_1 \log(pib_hab) + u$. Interpréter β_1 . Ajouter le carré de $\log(pib_hab)$ et calculer le revenu de retournement correspondant, $\exp(-\beta_1/(2\beta_2))$. Commenter.
- (vi) Au vu de ces résultats, peut-on conclure que la croissance économique finira par réduire les émissions de CO2 ? Discuter au moins trois limites de cette analyse.

Rendu

Compiler le document complet. Il doit se compiler sans erreur après un redémarrage de R (`Session > Restart R`). Déposer le fichier `TP5.html` sur CEL.

Troisième partie

SIG

Introduction

Le cours en résumé :

- 24 h
- objectifs : appréhender les données géographiques et maîtriser les principales opérations sur ce type de données (intersection, distance etc)
- Langage : R & QGIS (introduction)
- modalités d'examens :
 - Note de participation
 - Un projet commun SIG/économétrie avec choix du sujet imposé
 - Présentation en classe d'une carte réalisée avec QGIS : 3mn de présentation

Le plan :

1. Introduction à QGIS
2. Traitement sur les données vectorielles
3. Les SIG avec R

Toutes les données des TP sont disponibles sur [ce drive](#).

TP1 : Introduction à QGIS

1. Prise en main

1.1 Affichage / désaffichage des panneaux

- Fermez les panneaux **Couches** et **Identifier les résultats**.
- Affichez-les à nouveau avec le menu **Vue > Panneaux**.

1.2 Utilisation de données vecteur

Toutes les données des TP sont disponibles sur [ce drive](#).

1. Examinez la liste des fichiers du répertoire **Data/ADMIN EXPRESS**.
2. Affichez :
 - `ARRONDISSEMENT.shp`
 - `EPCI.shp`
 - `communes_ara.gpkg`
3. Ouvrez la table attributaire de la couche **ARRONDISSEMENT** :
 - Trier selon la colonne `INSEE_DEP`.
 - Quels sont les arrondissements du département de la Loire (42) ?
4. Supprimez la couche **ARRONDISSEMENT**.
5. Ouvrez les propriétés de la couche **COMMUNE** :
 - Notez le système de coordonnées, la géométrie, la liste des attributs.
6. Ouvrez la table attributaire de la couche **COMMUNE** :
 - Quel est le nombre de communes ?

7. Identifiez la commune de **Saint-Maurice-en-Gourgois** :
 - Dans quel département est-elle ?
 - Chargez la couche `DEPARTEMENT.shp` et identifiez le département 43.
 - Quelle est sa population (champ `POPULATION`) ?

1.3 Création d'un projet

- Enregistrez le projet sous le nom `TP1-1.qgs`.

1.4 Utilisation de l'outil Identifier les entités

- Utilisez l'outil sur Saint-Maurice-en-Gourgois : quelle est la longueur du périmètre ?
- Zoomez sur la dalle.

1.5 Jointure 1

- Joindre la couche **EPCI** à **COMMUNE**.
- Quel est le nom de l'EPCI de Saint-Maurice-en-Gourgois ?

1.6 Utilisation d'OpenStreetMap

1. Créez une connexion XYZ :
 - Nom : `OpenStreetMap`
 - URL : `https://tile.openstreetmap.org/{z}/{x}/{y}.png`
2. Installez l'extension **QuickMapServices**.
3. Affichez le fond de carte OSM Standard.

1.7 Ordre des couches et opacité

- Classez les couches dans l'ordre : Communes → EPCI → Arrondissement.

1.8 Groupe de couches

- Groupez ARRONDISSEMENT et COMMUNE en **ADMIN**.

1.9 Outil de mesure

- Mesurez la distance maximale de Saint-Maurice-en-Gourgois.

1.10 Sélection et export

- Sélectionnez les communes d’Auvergne-Rhône-Alpes.
- Exportez dans `CommunesEPCI_ARA.gpkg`.
- Créez une couche des seuls EPCI d’Auvergne-Rhône-Alpes.

1.11 Sélection et conditions multiples

- Communes ARA > 1000 habitants → combien ?
- Communes Haute-Loire > 1000 habitants → combien ?
- Exportez les deux sélections.

2. Symbologie 1 : Pays du monde

1. Téléchargez les données physiques et culturelles de **NaturalEarth** à 50m sur [ce lien](#)
2. Ouvrez :

- `ne_50m_admin_0_countries`
- `ne_50m_populated_places_simple`
- `ne_50m_geographic_lines`

3. Avec la première couche, créez une symbologie par PIB du pays (`GDP_MD`) en prenant une graduation Jenks.

4. Utilisez palette **Viridis**, projection **World Robinson (EPSG:54030)**.
5. Renommez la couche : *Countries by GDP*.
6. Enregistrez le projet : TP1_2.qgz.

2.1 Mise en page

- Créez une mise en page `gpd` en A4 paysage.
- Ajoutez carte, légende, fond gris clair, export en PNG 300 dpi.

2.2 Rendu gradué : Villes du monde

1. Utilisez `pop_max`.
2. Créez 6 classes (Jenks ou seuils manuels).
3. Symbole ponctuel rose, transparence 60 %, taille 0.5–4 mm.
4. Exportez en PNG 300 dpi.

2.2 Symboles proportionnels

- Symbole unique, rose, transparence 60 %.
- Taille proportionnelle `pop_max`.
- Enregistrez le projet.

3. Symbologie 2 : France

Ouvrir :

- `liste_cheflieu.geojson`,
- `occitanie_communes.gpkg`,
- `occitanie_limites.gpkg`,
- `liste-des-gares.geojson`,
- `RéseauFerré.gpkg`

Dans cette partie, il faut faire une carte de l'Occitanie en utilisant les 5 jeux de données

3.1 Styles

- Régions en gris clair + bordures blanches.
- Chefs-lieux : symboles catégorisés (`Préfecture région` et `Préfecture`).
- Réseau ferré : catégorisé sur `type_voie`.
- Communes Occitanie : densité de population (Jenks, OrRd, bornes manuelles).

3.2 Étiquettes

- Chefs-lieux avec règles selon statut administratif.

3.3 Mise en page finale

- A4 portrait, titre *Réseau ferré en Occitanie*, carte 1/2 000 000, légende, échelle, nord, sources.
- Ajoutez une carte miniature France + Occitanie.
- Export PNG et PDF.

TP2 : Traitement sur les données vectorielles

1. Données de départ

Dans un nouveau projet, ouvrez les couches :

- `departement_occitanie.gpkg`
- `CLC12_RLRMP_RGF.shp`

Pour visualiser les types d'occupation du sol avec les couleurs standard **Corine Land Cover**, ouvrez les propriétés de la couche et chargez le fichier de style `CLC12.sld` (dans le dossier *FichiersLegende*).

Ajoutez aussi :

- `trace-du-reseau-autoroutier-doccitanie.geojson`
- `dreal-occitanie-mats-eoliens.geojson`

Enregistrez le projet sous le nom `TP2.qgz`.

2. Création d'un GeoPackage

Toutes les couches produites seront enregistrées dans une base unique **GeoPackage**.

1. Créez `TP2_couches.gpkg` (répertoire *data*) en exportant la couche des mâts éoliens.
2. Options :
 - `format = GeoPackage`
 - `nom fichier = TP2_couches.gpkg`
 - `nom couche = Mâts éoliens`
 - `SCR = EPSG:2154`

3. Zone tampon sur l'autoroute A61

3.1 Vérification du SCR

Dans les propriétés de `trace-du-reseau-autoroutier-doccitanie`, identifiez le système de coordonnées.

3.2 Conversion en Lambert 93

Exportez la couche vers `TP2_couches.gpkg` avec :

- nom couche = Réseau autoroutier
- SCR = EPSG:2154

3.3 Sélection et tampon

1. Ouvrir la table attributaire → champ `num_route`.
2. Sélectionner les tronçons correspondant à **A61**.
3. Créer un tampon de **5000 m** avec options :
 - entités sélectionnées uniquement = Oui
 - Nb segments = 10
 - extrémités = Rond
 - regrouper = Oui
 - sortie = `TP2_couches.gpkg` → Tampon A61 5000m

3.4 Analyse

Avec **Compter les points dans les polygones**, combien d'éoliennes se trouvent dans cette zone tampon ?

4. Matrice de distance

Pour chaque mât éolien, calculer la distance avec les **2 plus proches voisins**.

- entrée = Mâts éoliens
- identifiant = id_mat
- type = *Matrice de distance linéaire* $(Nk+3)^*$
- k = 2
- sortie = TP2_couches.gpkg → Calcul Eoliennes 2 voisins

Inspectez le résultat avec l'outil Identifier : remarquez-vous le type de géométrie ?

5. Grille hexagonale

5.1 Création

1. Dans `departement_occitanie`, sélectionnez l'Aude et zoomez.
2. Avec **Créer une grille** :
 - type = hexagonale
 - étendue = canevas
 - espacement = 5000
 - SCR = EPSG:2154
 - sortie = TP2_couches.gpkg → Grille Aude 5km

5.2 Nettoyage

Supprimez les hexagones hors de l'Aude :

- sélection par localisation → inverser → supprimer en mode édition.

6. Comptages dans la grille

- **Compter points/polygones** → nb d'éoliennes par maille hexagonale.
 - sortie = TP2_couches.gpkg → Calcul nb éoliennes
- **Somme longueurs lignes** → total autoroutes par maille.

— sortie = TP2_couches.gpkg → Calcul long autoroutes

7. Analyse de superposition (Corine Land Cover)

7.1 Sélection des forêts

Sélectionner dans CLC12_RLRMP_RGF les polygones dont CODE_12 commence par “3”.
Exporter vers TP2_couches.gpkg → CLC12 Forêts Milieux SemiNat.

7.2 Superposition

Avec **Analyse de superposition** :

- source = Grille Aude 5km
- superposition = CLC12 Forêts Milieux SemiNat
- sortie = TP2_couches.gpkg → Calcul P ForêtsSemiNat

8. Intersection et Group Stats

8.1 Zone tampon des parcs

Créer tampon **2000 m** autour de chaque mât → Tampon Eol 2000m.
Puis regrouper par id_parc, n_parc → Calcul ParcEol 2000m.

8.2 Intersection avec CLC

Avec **Intersection** :

- source = CLC12_RLRMP_RGF
- superposition = Tampon Eol 2000m
- champs conservés : ID, CODE_12, id_parc, n_parc
- sortie = TP2_couches.gpkg → Calcul Inter ParcEol CLC12

8.3 Tableau croisé dynamique

Dans l'extension **Group Stats** :

- Couches = Calcul Inter ParcEol CLC12
- Colonnes = CODE_12
- Lignes = id_parc, n_parc
- Valeurs = Surface (somme)

Exporter le tableau et coller dans Excel/Calc.

TP3 : SIG avec R

Chargez les libraries suivantes :

```
#install.packages("spDataLarge", repos = "https://geocompr.r-universe.dev")
#install.packages("remotes")
#remotes::install_github("r-tmap/tmap")

library(tidyverse)
library(sf)
library(stars)
library(terra)
library(spData)
library(spDataLarge)
library(tmap)
library(leaflet)

#remotes::install_github("r-tmap/tmap")
```

1. Premières cartes

1. Décrire l'objet Utilisez `world`. Utiliser `summary()` sur la colonne de géométrie de l'objet `world` inclus dans le package `spData`. Utilisez `ggplot2`. Tracez la carte des continents. Utiliser le `theme_void()`. Tracez le continent asiatique, en filtrant puis appliquant la fonction d'union de formes géométrique `st_union`.
2. Rajouter à la carte des continents des ronds pour chaque pays représentant la racine carrée de leur population divisé par 10000. Il faudra pour ça calculer les centroides de chaque pays avec la commande `st_centroid` du package `sf`
3. Tracez la carte de l'Inde dans le continent asiatique. Il faut :
 - filtrer et tracer le continent asiatique dans `world` .
 - créer un objet `india` qui filtre l'Inde dans `world` .
 - rajouter la carte de l'Inde en grisant son contour

- créer le centroïde de l'Inde et rajouter sur la carte une étiquette “Inde” à la coordonnée du centroïde du pays
- 4. Créer un raster de 10x10 pixels avec la commande `rast`, dont les niveaux avec des valeurs aléatoires allant de 0 à 10 (avec la commande `runif`). Tracez ce raster avec `geom_raster`.
- 5. Chargez le fichier `raster/nlcd.tif` du package `spDataLarge` à l'aide de la commande. Décrivez cet objet. Utilisez la fonction `plot`. Enfin, convertissez le raster en objet du package `stars` et décrivez le résultat.

2. Opérations sur les attributs

Pour ces exercices, nous utiliserons les ensembles de données `us_states` et `us_states_df` du package `spData`.

1. Créez un nouvel objet appelé `us_states_name` qui contient uniquement la colonne `NAME` de l'objet `us_states` en utilisant la syntaxe de base R (`[]`) ou tidyverse (`select()`). Quelle est la classe du nouvel objet et qu'est-ce qui le rend géographique ?
2. Sélectionnez les colonnes de l'objet `us_states` contenant les données de population. Obtenez le même résultat en utilisant une autre commande (bonus : essayez de trouver trois façons d'obtenir le même résultat). Indice : essayez d'utiliser des fonctions d'aide, telles que `contains` ou `matches` de `dplyr` (voir `?contains`).
3. Trouvez tous les États ayant les caractéristiques suivantes (bonus : trouvez-les *et* affichez-les) :
 - Appartiennent à la région Midwest.
 - Appartiennent à la région Ouest, ont une superficie inférieure à 250 000 km² *et* en 2015, une population supérieure à 5 000 000 d'habitants (astuce : vous devrez peut-être utiliser la fonction `units::set_units()` ou `as.numeric()`).
 - Appartiennent à la région Sud, avaient une superficie supérieure à 150 000 km² ou une population totale en 2015 supérieure à 7 000 000 d'habitants.
4. Quelle était la population totale en 2015 dans l'ensemble de données `us_states` ? Quelle était la population minimale et maximale en 2015 ?
5. Combien d'États y a-t-il dans chaque région ?
6. Quelle était la population minimale et maximale en 2015 dans chaque région ? Quelle était la population totale en 2015 dans chaque région ?

7. Ajoutez des variables de `us_states_df` à `us_states` et créez un nouvel objet appelé `us_states_stats`. Quelle fonction avez-vous utilisée et pourquoi? Quelle variable sert de clé dans les deux ensembles de données? Quelle est la classe du nouvel objet?
8. `us_states_df` a deux lignes de plus que `us_states`. Comment pouvez-vous les trouver? (astuce : essayez d'utiliser la fonction `dplyr::anti_join()`)
9. Quelle était la densité de population en 2015 dans chaque État? Quelle était la densité de population en 2010 dans chaque État?
10. Combien la densité de population a-t-elle changé entre 2010 et 2015 dans chaque État? Calculez le changement en pourcentage et cartographiez-le.
11. Changez les noms des colonnes dans `us_states` en minuscules. (Astuce : les fonctions d'aide - `tolower()` et `colnames()` peuvent aider.)
12. Utilisez `us_states` et `us_states_df` pour créer un nouvel objet appelé `us_states_sel`. Le nouvel objet ne doit contenir que deux variables - `median_income_15` et `geometry`. Changez le nom de la colonne `median_income_15` en `Income`.
13. Calculez le changement du nombre de résidents vivant en dessous du seuil de pauvreté entre 2010 et 2015 pour chaque État. (Astuce : voir `?us_states_df` pour la documentation sur les colonnes du seuil de pauvreté.) Bonus : Calculez le changement en pourcentage des résidents vivant en dessous du seuil de pauvreté dans chaque État.
14. Quelle était la population minimale, moyenne et maximale des personnes vivant en dessous du seuil de pauvreté en 2015 pour chaque région? Bonus : Quelle est la région où l'augmentation du nombre de personnes vivant en dessous du seuil de pauvreté est la plus importante?
15. Créez un raster `grain` vide avec neuf lignes et colonnes et une résolution de 0,5 degré décimal (WGS84). Remplissez-le avec des nombres aléatoires. Extraire les valeurs des quatre cellules de coin.
16. Quelle est la classe la plus courante de notre exemple de raster `grain`?
17. Tracez l'histogramme et la boîte à moustaches du fichier `dem.tif` du package `spDataLarge` (`system.file("raster/dem.tif", package = "spDataLarge")`).

3. Opération sur les données spatiales

3.1 Opérations sur les vecteurs

1. Utiliser les jeux de données `nz` et `nz_height` du package `spData`. Combien de ces points élevés la région de Canterbury contient-elle?

Bonus : tracez le résultat en utilisant la fonction `plot()` pour montrer toute la Nouvelle-Zélande, la région de `Canterbury` en jaune, les points élevés à Canterbury représentés par des croix rouges (astuce : `pch = 7`) et les points élevés dans d'autres parties de la Nouvelle-Zélande

représentés par des cercles bleus. Consultez la page d'aide `?points` pour plus de détails avec une illustration des différentes valeurs `pch`.

2. Quelle région a le deuxième plus grand nombre de points `nz_height`, et combien en a-t-elle ?
3. En généralisant la question à toutes les régions : combien des 16 régions de Nouvelle-Zélande contiennent des points qui appartiennent aux 100 points les plus élevés du pays ? Quelles sont ces régions ?

Bonus : créez un tableau listant ces régions par ordre du nombre de points et leur nom.

4. Le point de départ de cet exercice est de créer un objet représentant l'État du Colorado aux États-Unis. Faites ceci avec la fonction `filter()` (`tidyverse`) et tracez l'objet résultant dans le contexte des États-Unis.
- Créez un nouvel objet représentant tous les États qui se chevauchent géographiquement avec le Colorado et tracez le résultat (astuce : la manière la plus concise de le faire est avec la méthode de sous-ensemble `[]`).
- Créez un autre objet représentant tous les objets qui touchent (ont une frontière commune avec) le Colorado et tracez le résultat (astuce : souvenez-vous que vous pouvez utiliser l'argument `op = st_intersects` et d'autres relations spatiales lors des opérations de sous-ensemble spatial en R de base).

Bonus : créez une ligne droite allant du centroïde du district de Columbia près de la côte Est au centroïde de la Californie près de la côte Ouest des États-Unis (astuce : les fonctions `st_centroid()`, `st_union()` et `st_cast()` peuvent aider) et identifiez quels États cette longue ligne est.

3.2 Opérations sur les rasters

5. Utilisez `dem = rast(system.file("raster/dem.tif", package = "spDataLarge"))`, et reclassifiez l'élévation en trois classes : basse (<300), moyenne et haute (>500). Ensuite, lisez le raster NDVI (`ndvi = rast(system.file("raster/ndvi.tif", package = "spDataLarge"))`) et calculez la moyenne du NDVI et de l'élévation pour chaque classe d'altitude.
6. Appliquez un filtre de détection de lignes à `rast(system.file("ex/logo.tif", package = "terra"))`. Tracez le résultat. Astuce : Lisez `?terra::focal()`.
- 7 Calculez l'indice d'eau normalisé (NDWI; $(\text{green} - \text{nir})/(\text{green} + \text{nir})$) d'une image Landsat. Utilisez l'image Landsat fournie par le package `spDataLarge` (`system.file("raster/landsat.tif", package = "spDataLarge")`). Calculez également une corrélation entre le NDVI et le NDWI pour cette région (astuce : vous pouvez utiliser la fonction `layerCor()`).

8. Un message sur [StackOverflow](#) montre comment calculer les distances jusqu'à la côte la plus proche en utilisant `raster::distance()`. Essayez de faire quelque chose de similaire mais avec `terra::distance()` : récupérez un modèle numérique d'élévation de l'Espagne et calculez un raster qui représente les distances jusqu'à la côte à travers le pays (astuce : utilisez `geodata::elevation_30s()`). Convertissez les distances résultantes de mètres en kilomètres. Note : il peut être judicieux d'augmenter la taille de cellule du raster d'entrée pour réduire le temps de calcul lors de cette opération (`aggregate()`).
9. Essayez de modifier l'approche utilisée dans l'exercice ci-dessus en pondérant le raster de distance avec le raster d'élévation ; chaque tranche de 100 mètres d'altitude devrait augmenter la distance jusqu'à la côte de 10 km. Ensuite, calculez et visualisez la différence entre le raster créé en utilisant la distance euclidienne (E7) et le raster pondéré par l'élévation.

4. Opérations sur les géométries

4.1 Opérations sur les vecteurs

1. Générez et tracez des versions simplifiées de l'ensemble de données `nz`. Expérimentez avec différentes valeurs de `keep` (allant de 0,5 à 0,00005) pour `ms_simplify()` et `dTolerance` (de 100 à 100 000) pour `st_simplify()`.
 - À partir de quelle valeur le résultat commence-t-il à se détériorer pour chaque méthode, rendant la Nouvelle-Zélande méconnaissable ?
 - Avancé : Quelle est la différence entre le type de géométrie des résultats de `st_simplify()` par rapport au type de géométrie de `ms_simplify()` ? Quels problèmes cela crée-t-il et comment cela peut-il être résolu ?
2. Dans le premier exercice du chapitre sur les opérations de données spatiales, il a été établi que la région de Canterbury avait 70 des 101 points les plus élevés de Nouvelle-Zélande. En utilisant `st_buffer()`, combien de points dans `nz_height` se trouvent à moins de 100 km de Canterbury ?
3. Trouvez le centroïde géographique de la Nouvelle-Zélande. À quelle distance se trouve-t-il du centroïde géographique de Canterbury ?
4. La plupart des cartes du monde ont une orientation nord en haut. Une carte du monde avec une orientation sud en haut pourrait être créée par une réflexion (l'une des transformations affines non mentionnées dans ce chapitre) de la géométrie de l'objet `world`. Écrivez le code pour le faire. Astuce : vous devez utiliser un vecteur à deux éléments pour cette transformation. Bonus : créez une carte à l'envers de votre pays.
5. Calculez la longueur des lignes de frontières des États-Unis en mètres. Quel État a la frontière la plus longue et lequel a la frontière la plus courte ? Astuce : La fonction `st_length` calcule la longueur d'une géométrie de type `LINestring` ou `MULTILINESTRING`. Il

faut aussi transformer la géométrie avec un CRS qui puisse calculer des distances : ici ESPG=2163.

6. Exécutez le code de la section 5.2.6. En référence aux objets créés dans cette section, sélectionnez le point dans `p` qui est contenu à la fois dans `x` et `y`.
 - Utilisez les opérateurs de sous-ensemble de base.
 - Utilisez un objet intermédiaire créé avec `st_intersection()`.
6. Exécutez le code suivant :

```
bb = st_bbox(st_union(x, y))
box = st_as_sfc(bb)
set.seed(2024)
p = st_sample(x = box, size = 10)
p_xy1 = p[x_and_y]
plot(box, border = "gray", lty = 2)
plot(x, add = TRUE, border = "gray")
plot(y, add = TRUE, border = "gray")
plot(p, add = TRUE, cex = 3.5)
plot(p_xy1, cex = 5, col = "red", add = TRUE)
text(x = c(-0.5, 1.5), y = 1, labels = c("x", "y"), cex = 3)
```

En référence aux objets créés dans cette section, sélectionnez le point dans `p` qui est contenu à la fois dans `x` et `y`.

- Utilisez les opérateurs de sous-ensemble de base.
- Utilisez un objet intermédiaire créé avec `st_intersection()`.

4.2 Opérations sur les rasters

7. Lisez le fichier `srtm.tif` dans R (`srtm = rast(system.file("raster/srtm.tif", package = "spDataLarge"))`). Ce raster a une résolution de 0,00083 par 0,00083 degrés. Modifiez sa résolution en 0,01 par 0,01 degrés en utilisant toutes les méthodes disponibles dans le package `terra`. Visualisez les résultats. Pouvez-vous remarquer des différences entre les résultats de ces méthodes de rééchantillonnage ?

5. Application : rapprochement de base par distances

Le but de cet exercice est d'identifier la nature de stations de services. Un jeu de données issu de <https://www.data.gouv.fr/fr/datasets/prix-des-carburants-en-france-flux-instantane-v2-amelioree/> donne les prix des carburants mais on n'a pas d'information sur le type de station

(station d'autoroute, de supermarché...). Le jeu de données magasins d'openstreetmap pourrait permettre d'apporter des informations.

1. Charger les données de `TP3.zip`. Les gpkg s'ouvrent la commande `st_read` du package `sf`.
2. Restreindre la base `magasins` aux types de magasins (`shop`) suivants : “gas”, “supermarket”, “convenience”, “car_repair”, “car”, “mall”, “convenience:gas”
3. Transformer les deux jeux en sf dataframe en système de coordonnées EPSG 2154. Attention, pour la base `station`, il faut diviser longitude et latitude par 100000.
4. Pour chaque station, trouver le magasin le plus proche et calculer la distance correspondante.
5. Quelle est la part des magasins à moins de 100 mètres d'une station.
6. Ajouter les attributs `shop` et `operator` pour chaque magasin le plus proche à la base stations.